1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA
DEPARTAMENTO DE ENGENHARIA ELECTROTÉCNICA E DE COMPUTADORES

ADVANCED COMPUTER VISION / AI FOR COMPUTER VISION

ASSIGNMENT #2

2026/27 Edition

Object Recognition and Image Retrieval

From handcrafted descriptors to self-supervised and vision–language representations

Advanced Computer Vision / AI for Computer Vision
Due Date : November 8th, 2026

Main objective

Build and evaluate image classifiers and retrieval systems using handcrafted local features, pretrained visual representations, and image–text similarity. Analyze accuracy, data efficiency, robustness and computational cost using a common evaluation protocol.

Code and data are available at the following link:

Access the code and data repository

Please read the README.md file carefully for detailed instructions, implementation guidelines, and relevant information on the provided resources.

Overview and learning objectives

Image classification assigns an image to one or more semantic categories. Image retrieval ranks a collection of images according to how likely they are to contain a target category. This assignment follows the original progression from local descriptors and histogram encodings to pretrained neural representations, while introducing modern self-supervised and vision–language models.

The core categories are airplanes, motorbikes, people, horses and cars, subject to the availability of the provided dataset. Use the same fixed data splits across methods wherever possible.

Learning objectives

- Construct and interpret image representations based on SIFT, Bag of Visual Words (BoVW), VLAD and Fisher Vectors.
- Train linear classifiers and understand the effects of normalization, spatial information, training-set size and regularization.
- Extract frozen features from a pretrained CNN or Vision Transformer (ViT), including DINOv2.
- Compare supervised classification using labeled examples with CLIP zero-shot classification based on image–text similarity.
- Design and evaluate image retrieval systems using local descriptors (BoVW, VLAD and Fisher Vectors), pretrained visual embeddings (CNNs and DINOv2), and text-image similarity (CLIP).
- Evaluate classification and retrieval with precision, recall, Average Precision (AP), mean Average Precision (mAP), and Precision@K.
- Design controlled experiments, avoid test-set leakage, and report accuracy alongside computational and storage costs.
- Analyze errors, prompt sensitivity, background bias and the limitations of visual explanations.

General experimental protocol

Data splits and fairness

- Use the supplied training, validation and test partitions. If no validation partition is supplied, create one from the training data before any model selection.
- Do not use the test set to choose hyperparameters, prompts, thresholds, training fractions or preprocessing settings. Use it once for final reporting.
- Use identical test images and ground-truth labels for all compared methods.
- For small-data experiments, repeat random subsampling with at least three seeds where practical and report mean ± standard deviation.
- Record the software versions, model identifier, pretrained weights, image preprocessing, random seeds and hyperparameters.

Metrics

For binary one-vs-rest retrieval, report the precision–recall curve and Average Precision (AP). For multiple categories, report per-class AP and their mean (mAP). Also report a confusion matrix and precision/recall at a stated operating threshold for classification. Use Precision@K and, where useful, Recall@K for ranked retrieval. Define the positive class and the convention used for AP.

Record feature dimensionality, feature extraction time, classifier training time, inference time and approximate storage where feasible. State the hardware and whether timings include preprocessing and model loading.

Part 1 — Handcrafted features: BoVW and linear SVM

This part used Handcrafted features an is based on the use of dense SIFT, visual vocabularies and spatial histograms. Use the provided descriptors and code where available, or implement the corresponding pipeline in Python.

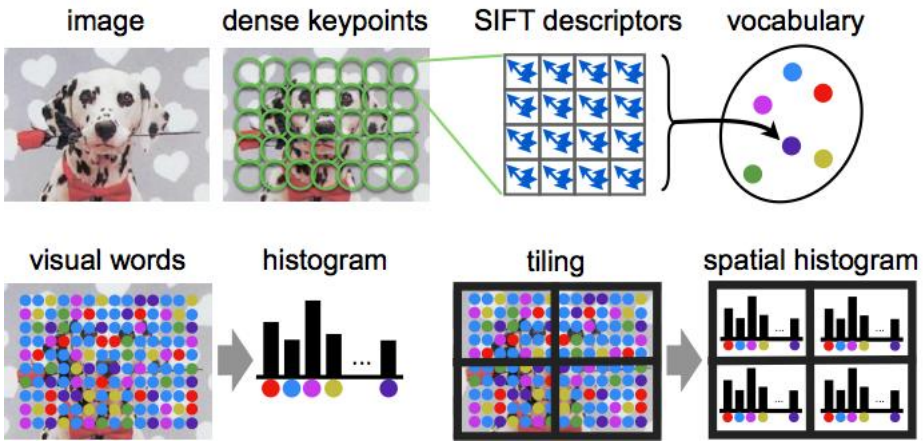
Stage A — Prepare the data and build BoVW

The 'data' directory comprises of images and pre-computed descriptors for each image. The JPEG images are stored in the 'data/images' directory. The data contains three distinct image classes, namely airplanes, motorbikes, and persons, along with a collection of background images that do not belong to any of these classes. During the data preparation stage, this data is partitioned as follows:

	aeroplane	motorbike	person	background
training	112	120	1025	1019
test	126	125	983	1077
total	238	245	2008	2096

An image descriptor is derived from the statistical analysis of local image features, which are representative of the visual attributes of various local image regions. Typically, the process of mapping local features to a single descriptor vector is considered an encoding step, and the resulting descriptor is often referred to as a code. The significant advantage of using codes over sets of local features lies in the fact that codes can be compared using simple vectorial metrics, such as the Euclidean distance. This feature also makes them much more convenient to employ when learning an image classifier.

This initial part will focus on using the Bag-of-Visual-Words (BoVW) encoding method. The process of generating a BoVW descriptor from an image can be summarized as follows:



1. Compute SIFT (Scale-Invariant Feature Transform) features on a regular grid across the image (also known as 'dense SIFT').
2. Vector quantize the SIFT features into visual words using a clustering algorithm such as k-means.
3. Count the frequency of each visual word in the image and record it in a histogram.
4. Divide the image into spatial tiles and repeat steps 2 and 3 for each tile.
5. Concatenate the histograms of all tiles to form a final feature vector for the image.

This process results in a fixed-length representation of the image that can be used for various computer vision tasks such as image classification, retrieval, and object recognition.

- Inspect representative images from the target categories and background/negative examples. Note variation in object scale, viewpoint, illumination, clutter and background.
- Compute dense SIFT descriptors (or use the provided precomputed descriptors). Learn a visual vocabulary with k-means using training data only.
- Construct BoVW histograms. Compare a global histogram with a spatial-pyramid representation that concatenates histograms from image regions.
  NOTE: In order to provide an easy path for you to start this assignment, we provided an initial set of descriptors : k=512 for BoVW, k=32, d=80 (SIFT+PCA) for VLAD, k=32, d=2x80 (SIFT+PCA) for FV and 2x2 image tilling;
- Evaluate at least two additional vocabulary sizes, for example K = 256 and K = 1,024. Keep the train/validation/test protocol unchanged.

  Q: How do you choose the number of visual words for BoVW?

  Q: How does spatial tiling encode useful layout information? When might it reduce generalization?

  Q: How does vocabulary size affect representation capacity, quantization error, computation and storage?

Stage B — Train a motorbike classifier

- Use motorbike images as positive samples and the designated background images as negatives. Train a linear SVM on the BoVW descriptors.
- Rank training images by the classifier's decision score and inspect high- and low-ranked examples.
- Visualize the visual words or image regions associated with high classifier weights, using the supplied function (*RelevantVisualWordsDisplay)* to supports this.

  Q: What do you anticipate the results to be? What do you predict will occur?

  Q: Do the highest-ranked images share object features, contextual cues or background patterns?

  Q: Why can a classifier achieve high training performance but fail on new images?

Stage C — Test and evaluate retrieval

- Apply the trained classifier to the held-out test set. Rank images by the continuous SVM decision score.
- Plot the precision–recall curve and calculate AP. Report the result for the target category.

  Q: Why is the bias term irrelevant to the ordering of images scored by one fixed binary linear classifier?

  Q: How does the ordering of positive and negative examples affect the precision–recall curve and AP?

Stage D — Other categories

- Repeat the experiment for airplanes, people, horses and cars, as supported by the dataset. Keep the split and evaluation protocol consistent, and record per-class AP.

Q: How do intra-class variation, sample count, object scale and background diversity help explain differences in AP?

Stage E — Normalization and spatial information

Normalization is a crucial practical aspect of image descriptors. For instance, in the case of BoVW descriptors, which are histograms, it is reasonable to expect that their elements should sum to 1 if we treat them as discrete probability distributions. This implies normalizing the BoVW descriptor vectors using the L1 norm. However, in most cases L2 norm is used.

A linear SVM can be thought of as using a linear kernel

$$K(\mathbf{h}, \mathbf{h}') = \sum_{i=1}^{d} h_i h_i'$$

to measure the similarity between pair of objects h and h' (in this case pairs of BoVW descriptors).

- Compare no normalization, L1 normalization and L2 normalization of BoVW histograms. Select settings using the validation set.
- Compare spatial-pyramid histograms against a single global histogram.

Q: How does L2 normalization affect the self-similarity of a nonzero feature vector under a linear kernel?

Q: Regarding the L2 normalized BoVW histogram $\mathbf{h}$, what can be inferred about its self-similarity $K(\mathbf{h}, \mathbf{h})$?

Q: How does the self-similarity $K(\mathbf{h}, \mathbf{h})$ of an L2 normalized BoVW histogram $\mathbf{h}$ compare to the similarity $K(\mathbf{h}, \mathbf{h}')$) of two distinct L2 normalized BoVW histograms $\mathbf{h}$ and $\mathbf{h}'$?

Q: Would the same hold true for unnormalized or L1 normalized histograms?

Q: Is there a correlation between classification performance and the use of L2 normalization?

Q: Why can removing spatial information help some classes but hurt others?

Stage F — Hellinger feature map

Thus far, a linear SVM has been employed, treating the image histograms as vectors with a Euclidean norm of one. We will now utilize a Hellinger kernel classifier, but rather than computing kernel values, we will explicitly compute the feature map, ensuring that the classifier remains linear in the new feature space. For nonnegative, L1-normalized histograms $\mathbf{h}$ and $\mathbf{h}'$, the Hellinger kernel (also referred to as the Bhattacharyya coefficient) is defined as follows:

$$K(\mathbf{h}, \mathbf{h}') = \sum_{i=1}^{d} \sqrt{h_i h_i'}, \quad \mathbf{h} = \begin{bmatrix} h_1 \\ \vdots \\ h_d \end{bmatrix}, \quad \mathbf{h}' = \begin{bmatrix} h_1' \\ \vdots \\ h_d' \end{bmatrix}$$

Compare this expression with that of the linear kernel given earlier. In calculating the feature map, only the square root of the histogram values is required.

Q: Considering the rule of thumb mentioned earlier that suggests L2 normalization can improve the performance of linear SVMs, how should the BoVW histograms $\mathbf{h}$ and $\mathbf{h}'$ be normalized for the Hellinger kernel classifier? Is it recommended to perform the normalization before or after taking the square root of the histogram values?

Q: Why does the square-root mapping change the relative contribution of frequent and rare visual words?

We will now train the image classifier using the Hellinger kernel instead of the linear one.

- Edit *your code* to implement the Hellinger kernel by taking the square root of the histograms for feature vectors and applying the appropriate normalization. This will require adding one line of code for both the training and test histograms in python.
- Retrain the classifier for the motorcycle class using the Hellinger kernel and the square root of the histograms as feature vectors. Make sure to apply the appropriate normalization. Evaluate the performance of the classifier on the test dataset.
- Experiment with the other histogram normalization options and compare their performance with your chosen normalization method to ensure that your choice provides the best results.

Note: *To reduce the training time, we are not modifying the C parameter while learning the SVM. Typically, this parameter affects the generalization error and should be determined on a validation set when the kernel is modified. However, for this case, we can verify experimentally that the impact of C is relatively small.*

Stage G — Vary the number of training images

Train with increasing fractions of the available labeled data, for example 10%, 25%, 50% and 100%. Plot validation AP against the number of training examples. Repeat subsampling with several seeds when practical.

Q: What performance do you get with the linear kernel? And with the Hellinger kernel?

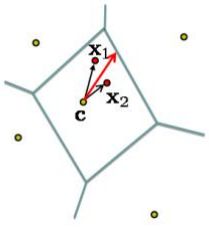
Q: Does performance saturate? Would collecting more examples or improving their diversity likely help more? Support your answer with the performance curve.

Part 2 — Advanced encodings: VLAD and Fisher Vectors

Up to this point we have used a BoVW representation of the original dense SIFT description of the image, together with spatial information coded by a spatial tiling of the image. BoVW records visual-word counts. VLAD additionally aggregates residuals from local descriptors to their assigned cluster centres. Fisher Vectors use a Gaussian mixture model and encode first- and second-order statistics, commonly with diagonal covariance.

Stage H — VLAD

The vector of locally aggregated descriptors (VLAD) is an example of a first order encoding. It records the residuals of vectors within each Voronoi cell (i.e. the difference between the SIFT vectors and the cluster centre (from the *k*-means) as illustrated below:



Suppose the SIFT descriptors have dimension D, then the total size of the VLAD vector is K x D (since a D-dimensional residual is recorded for each Voronoi cell). Typically, K is between 16 and 512, and D is 128 or less.

- Replace BoVW with VLAD while keeping the underlying local descriptors and data partitions consistent. *Optional*: Apply a documented normalization pipeline (for example, intra-normalization followed by global L2 normalization if implemented). Select settings on validation data.
- Replace the encoding used in Part I with the VLAD encoding, and repeat the classification and retrieval experiments for the selected categories of Part I.
- Note, the performance improvement obtained simply by changing the feature encoding.

Q: For K visual words and D-dimensional local descriptors, compare the nominal dimensions of BoVW (K) and VLAD (K × D). What changes in representation capacity and storage?

Stage I — Fisher Vector

The Fisher Vector (FV) is an example of second order encoding. It records both the residuals (as in VLAD) and also the covariance of the SIFTs assigned to each Voronoi cell. Its implementation uses a Gaussian Mixture Model (GMM) (instead of k-means) and consequently SIFTs are softly assigned to mixture components (rather than a hard assignment as in BoVW). Suppose there are K mixture components and the covariance is restricted to a diagonal matrix (i.e. only the variance of each component is recorded) then the total size of the Fisher vector is 2K x d.

- Fit a Gaussian mixture model on training descriptors only, then compute Fisher Vectors for the images.
- Replace the encoding used in Part I with the Fisher Vector encoding, and repeat the classification and retrieval experiments for the selected categories of Part I.
- Use a consistent classifier and data splits to compare Fisher Vector, VLAD and BoVW.
- Report AP/mAP, feature dimensions, extraction time and storage.

Q: What additional information does a Fisher Vector encode compared with VLAD, and what computational costs does it introduce?

Part 3 - Large visual vocabularies with PCA vs smaller vocabularies

A key question is whether a richer image representation built with a larger visual vocabulary, followed by PCA, is better than directly using a smaller vocabulary. Evaluate this question explicitly rather than assuming that either strategy is superior.

Why compare these strategies?

A larger codebook can represent more fine-grained visual patterns and reduce quantization ambiguity: descriptors that are visually different are more likely to be assigned to different clusters. However, it creates a higher-dimensional, sparser representation, increases codebook fitting and encoding cost, and may overfit when the training set is small. PCA can compress the resulting image descriptors, reduce redundancy, and potentially improve classifier efficiency. But PCA is a linear projection: it can discard low-variance dimensions that still carry class-discriminative information, and it does not eliminate the cost of learning or initially constructing the larger representation.

A smaller codebook is cheaper and often produces denser, simpler representations, but may merge distinct patterns into the same visual word. The outcome depends on dataset size, class similarity, descriptor type, normalization, and classifier.

Stage A : Experimental design

Use at least three vocabulary sizes (for example K = 256, 512, and 1024; adapt these values to the available compute). For each K, evaluate BoVW without PCA and, where meaningful, PCA-reduced variants. Choose PCA dimensions that permit a fair comparison, such as 128 and 256 dimensions when the original feature dimension is sufficiently large. Also include a small-codebook baseline whose feature dimension is close to the PCA output dimension. If the original BoVW histogram has K dimensions, PCA output dimension must be smaller than K and no greater than the number of available training samples minus one.

Condition	Codebook K	PCA	Purpose	Record
A	Small (e.g. 256)	No	Low-cost baseline	Accuracy/AP, time, dimension
B	Large (e.g. 1024)	No	High-capacity baseline	Accuracy/AP, time, dimension
C	Large (e.g. 1024)	Yes (e.g. 128/256)	Compress richer representation	Accuracy/AP, time, dimension
D	Match C's final dimension where possible	No	Compare against a compact vocabulary	Accuracy/AP, time, dimension

Important fairness note: for BoVW, the histogram dimension is exactly K. PCA can reduce that K-dimensional histogram. For VLAD and Fisher Vectors, the raw representation dimension can grow much faster with K, so PCA may have a larger practical effect. State the representation and dimension explicitly; do not compare feature dimensions as if BoVW, VLAD, and Fisher Vector dimensions were interchangeable.

Measurements and interpretation

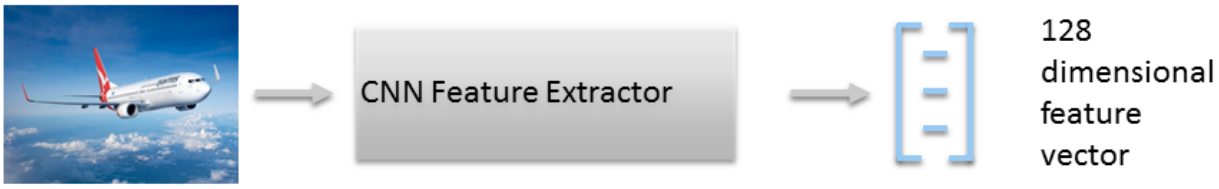
- Recognition: report test accuracy and macro-F1; also report mean Average Precision (mAP) if the task is evaluated as retrieval or one-vs-rest ranking.
- Compactness: report original and final feature dimensions and the percentage of dimensions removed by PCA.
- Efficiency: measure codebook fitting time, feature encoding time, PCA fitting time, classifier training time, inference time per image, and approximate feature storage.
- Stability: repeat the main comparison with at least three random seeds when practical, or use a fixed split and clearly acknowledge the limitation. não era viável a nível computacional
- Analysis: discuss whether PCA preserves performance, improves generalization, or harms discriminative information. Distinguish the cost of constructing the large representation from the lower cost of storing/classifying its reduced version.

Questions to answer in the report

Q: Does a larger codebook improve recognition before PCA? For which classes or visual patterns?
Q: Does PCA retain the gain from the larger codebook, and how much dimensionality reduction is possible before performance drops?
Q: At a similar final feature dimension, does a large codebook plus PCA outperform a directly small codebook?
Q: What is the trade-off between accuracy/mAP, memory, feature extraction time, and classifier inference time?
Q: Does the answer change for BoVW versus VLAD/Fisher Vector, and why might their dimensionality behave differently?

Part 4 — Pretrained visual representations: CNN, DINOv2 and CLIP

This part replaces the classifier dependency on the handcrafted feature extractors with a modern, reproducible feature-extraction workflow. The main experiments use frozen pretrained models; training a large backbone from scratch is not required.



Stage A — Frozen CNN features

- Choose a pretrained CNN (for example, ResNet-18) available through torchvision or another documented model library. Extract features from a specified layer (see below - Use global average pooling on each of the final 3 layers to extract descriptors).
- Train a linear SVM on frozen features and evaluate it on the same splits used in Part 1.
- Compare the results against BoVW and, if completed, VLAD/Fisher Vector.
- **Q:** How do learned features differ from handcrafted local descriptors? Does a higher-dimensional feature necessarily give better AP?

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Stage B — DINOv2 feature extraction

DINOv2 is a self-supervised family of Vision Transformer models. Its pretrained representations are learned from images without requiring human class labels for the pretraining task. For image-level classification, use the global image representation; patch tokens can be explored for spatial analysis.

- Load a pretrained DINOv2 model, such as ViT-S/14, and use the official model preprocessing. Keep the backbone frozen.
- Extract and cache the global image embedding for every train and test image. Normalize features consistently before fitting the classifier.

- Train a linear SVM and report per-class AP, mAP, confusion matrix, feature dimension, extraction time and inference cost.
- Optional extension: compare global image embeddings with aggregated patch-token features, explaining how aggregation affects spatial information.

Q: Why can DINOv2 features be useful with few labeled examples even though DINOv2 is not trained as a classifier for these target categories?

Q: What information might be lost when patch tokens are replaced by a single global descriptor?

Stage C — Few-shot learning and augmentation

- Using one selected pretrained representation, train the linear classifier with 1, 5, 10, 20 and as many positive examples as practical. State how negative examples were selected.
- Compare no augmentation with horizontal-flip augmentation. Apply augmentation only to training data unless test-time augmentation is explicitly being evaluated as a separate condition.
- Plot validation AP against the number of labeled examples. Report mean and standard deviation across multiple random subsets when practical.

Q: How much does pretrained representation quality reduce dependence on labeled examples?

Q: When could horizontal flipping harm recognition? Is averaging feature vectors equivalent to averaging classifier scores?

Stage D — Hyperparameter selection

- Evaluate a logarithmic range of SVM C values, such as 0.1, 1, 10, 100 and 1,000, using validation AP.
- Plot training and validation performance to diagnose overfitting. Select C using validation data, then evaluate the chosen configuration once on the test set.

Q: How does regularization affect the bias–variance trade-off when only a few positive examples are available?

Stage E — CLIP zero-shot classification

CLIP is trained to align image embeddings with text embeddings. Unlike DINOv2, it can directly compare an image with text prompts representing candidate categories, allowing zero-shot classification without training a target-specific classifier.

- Load a pretrained CLIP model and apply its official image and text preprocessing.
- Create candidate prompts for the five categories, for example "*a photo of a motorbike*" and "*a photo of a horse*". Encode each prompt and compute image–text cosine similarities.
- Predict the category with the highest similarity. Report the confusion matrix and per-class metrics. For class-wise retrieval, rank images by similarity to the relevant class prompt and compute AP/mAP.
- Compare a single prompt per category with a small set of alternative descriptions or prompt templates. Choose prompts using prior knowledge or validation data only; do not use test labels to tune prompts.

Q: How does CLIP zero-shot recognition differ from a linear SVM trained on frozen CLIP image embeddings?

Q: How sensitive are predictions to prompt wording? What are the risks of selecting prompts based on the test set?

Stage F — Visual analysis and failure cases

- Inspect correctly and incorrectly classified examples from the CNN, DINOv2 and CLIP experiments.
- Where feasible, visualize class-relevant regions using an appropriate method such as patch similarity, an activation map or a saliency method supported by the chosen architecture.

- Compare examples where the model focuses on the object with examples where it appears to rely on context or background.

  Q: Do the highlighted regions correspond to the object? What can and cannot be concluded from a saliency visualization?

  Q: Which categories and visual conditions remain difficult, and what evidence supports your explanation?

Part 5 — Open-ended image retrieval challenge

In this final part, design and evaluate an image retrieval system for a selected visual category using a fixed image corpus and a small set of query images. The objective is to rank the corpus images according to their relevance to the query and retrieve relevant images near the top of the ranking.

The system must operate as a retrieval pipeline: given a query image or text description, it computes similarity scores against the corpus and returns a ranked list of images. No classification model or classification-based decision is required.

Stage A — Collect examples and build a retrieval system

- Select a target category and construct a fixed image corpus containing relevant and irrelevant images. You may use the supplied dataset or collect additional images from permitted sources, documenting their origin and any applicable usage conditions.
- Define the ground truth for retrieval by identifying which corpus images are relevant to each query. Where possible, include visually similar but irrelevant images to make retrieval more challenging.
- Select a small, diverse set of query images, ensuring that the queries are not included in the corpus as exact duplicates.
- Divide the available data into query and corpus sets, avoiding duplicate or near-duplicate images across them.

Stage B — Implement and Compare Retrieval Strategies

Implement and compare at least two of the following retrieval strategies:

1. **Local-feature retrieval — BoVW**
   - Extract SIFT descriptors from the corpus images and build a visual vocabulary.
   - Represent each image as a normalized Bag-of-Visual-Words (BoVW) histogram.
   - Represent each query image in the same feature space.
   - Rank corpus images using an appropriate histogram similarity or distance measure, such as cosine similarity or histogram intersection.
2. **Aggregated local-descriptor retrieval — VLAD or Fisher Vector**
   - Represent each image using VLAD or Fisher Vector descriptors.
   - Apply appropriate normalization and, if computationally feasible, PCA dimensionality reduction.
   - Rank corpus images using cosine similarity or Euclidean distance.
   - Compare the retrieval performance and computational cost with BoVW.

3. Deep visual retrieval — CNN or DINOv2

- Extract global image embeddings using a pretrained CNN or DINOv2 model, without training a classification model.
- Normalize the embeddings and rank corpus images by cosine similarity.
- Optionally investigate patch-level features or image-region representations to determine whether they improve retrieval when images contain substantial background clutter or multiple objects.

4. Text-to-image retrieval — CLIP

- Encode a natural-language query describing the target visual concept using CLIP's text encoder.
- Encode the corpus images using CLIP's image encoder.
- Rank the corpus images according to image–text embedding similarity.
- Investigate whether changing the wording or level of detail in the text query changes the ranking.

Stage C — Evaluate Retrieval Performance

- Use the same fixed corpus and ground-truth relevance annotations for all methods.
- Evaluate retrieval using **Precision@K**, **Average Precision (AP)** and, where appropriate, **mean Average Precision (mAP)** across multiple queries.
- Report Precision@5, Precision@10, and Precision@20.
- Report Recall@K when the total number of relevant corpus images is known.
- Display representative top-ranked results, including both successful retrievals and failure cases.
- Measure feature extraction time, storage requirements, and query-time ranking cost.

Compare the retrieval methods in terms of ranking quality, robustness, efficiency and sensitivity to query selection.

Questions to address:

Q: Which retrieval strategy provides the best results for the selected category? Support your answer with quantitative metrics and representative retrieval examples.
Q: How does retrieval based on local visual patterns (BoVW or VLAD) compare with retrieval based on global deep embeddings (CNN or DINOv2)?
Q: Does increasing the visual vocabulary size improve retrieval quality? Can PCA reduce descriptor dimensionality while retaining retrieval performance?
Q: How does the diversity of query images affect retrieval performance? Compare results using different queries or query sets, including visually similar and substantially different examples.
Q: For CLIP, how sensitive is text-to-image retrieval to the wording of the query? Does a more detailed description improve the ranking?
Q: What are the limitations of evaluating retrieval on a small corpus? Discuss the effects of corpus size, ground-truth completeness, class imbalance, and near-duplicate images.

Required report and submission

Report structure

- Title, authors and a concise statement of the task.
- Introduction and background covering the representations and classifiers studied.
- Methods and implementation details, including data splits, preprocessing and hyperparameters.
- Results, with a consolidated comparison table and the required plots.
- Discussion of learning curves, errors, prompt sensitivity, computational cost and limitations.
- Conclusions identifying the best trade-offs and explaining the evidence.
- References for models, papers, datasets, software and reused code.

Required comparison table

Method	Supervision	AP / mAP	Feature dimension	Extraction cost	Inference cost
BoVW + SVM	Task labels	Report	Report	Report	Report
VLAD + SVM	Task labels	Report	Report	Report	Report
Fisher Vector + SVM	Task labels	Report	Report	Report	Report
CNN/ViT + SVM	Pretrained + task labels	Report	Report	Report	Report
DINOv2 + SVM	Self-supervised + task labels	Report	Report	Report	Report
CLIP zero-shot	Image–text pretraining; no task labels	Report	Report	Report	Report

Code and reproducibility

- Submit source code or notebooks, a README with execution instructions, dependency/environment information and references to reused code.
- Include scripts that reproduce the reported metrics and plots. Do not include large pretrained weights unless explicitly requested.
- Do not include private credentials or data that cannot legally be redistributed. Explain how permitted external data can be obtained.
- Make clear which experiments were completed and identify any deviations from the protocol.

Suggested grading rubric

Criterion	Weight	Evidence
Implementation and correctness	20%	Correct pipelines, clear code and reproducibility.
Experimental design	10%	Fair comparisons, sound splits and validation-only selection.
Quantitative evaluation	25%	Correct metrics, plots, tables and cost measurements.
Analysis and critical discussion	30%	Evidence-based interpretation, error analysis and limitations.
Report quality and references	15%	Clear organization, figures, citations and writing.

Software and practical notes

Recommended stack: Python 3, NumPy, scikit-learn, OpenCV, PyTorch and torchvision. Transformers or an official model implementation may be used for CLIP and DINOv2. Pin compatible versions in the environment file. The assignment should remain feasible without training a large model from scratch; cached descriptors and frozen pretrained features are encouraged.

If GPU resources are limited, use smaller pretrained models, process images in batches and cache extracted features. Make all comparisons on the same data splits and disclose differences in preprocessing.

References and resources

- Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. International Journal of Computer Vision.
- Jégou, H. et al. (2010). Aggregating Local Descriptors into a Compact Image Representation. CVPR. (VLAD)
- Perronnin, F., Sánchez, J. and Mensink, T. (2010). Improving the Fisher Kernel for Large-Scale Image Classification. ECCV.
- Oquab, M. et al. (2023). DINOv2: Learning Robust Visual Features without Supervision. https://arxiv.org/abs/2304.07193
- Radford, A. et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. ICML. https://arxiv.org/abs/2103.00020
- Official DINOv2 implementation: https://github.com/facebookresearch/dinov2
- OpenAI CLIP implementation: https://github.com/openai/CLIP

Implementation Guidelines

Code and data are available at the following link:

Access the code and data repository

The repository includes the dataset images and precomputed handcrafted feature representations using **Bag of Visual Words (BoVW), VLAD, and Fisher Vectors (FV)**.

Deep learning features must be extracted using the **ResNet-18** and **Vision Transformer (ViT)** architectures.

Please read the README.md file carefully for detailed instructions, implementation guidelines, and relevant information on the provided resources.